
ECE285: Deep Generative Model - Project Midterm Report

Liang Yang

Department of Electrical and Computer Engineering
University of California, San Diego
La Jolla, CA 92093
liy027@ucsd.edu

1 Problem Description & Significance

We aim to develop a **compute-efficient and few-shot personalization method for text-to-image diffusion models** that can faithfully capture a specific subject or visual style from only 10–30 images without sacrificing text controllability. Concretely, our problem is to inject a new concept (e.g., a person, product, mascot or brand style) into a pretrained Stable Diffusion-type model so that the model (1) preserves identity/style across diverse prompts and scenes, (2) generalizes beyond the limited poses/backgrounds in the tiny training set, (3) remains modular and reusable through small adapter weights (e.g., LoRA), and (4) trains stably and reproducibly via parameter-efficient fine-tuning plus lightweight regularization.

This problem matters because real creative and commercial workflows like design, advertising and creator tools need subject-consistent images on demand while facing scarce data and limited compute. Full-model fine-tuning is often impractical for students or small teams. *PEFT* approaches make state-of-the-art personalization accessible and affordable. Moreover, naïve few-shot tuning can overfit or erode prompt responsiveness, undermining reliability. Establishing a robust and low-cost recipe that balances faithfulness and generalization leads to faster and cheaper content production, and it integrates cleanly with modern control modules (ControlNet/IP-Adapter) for structured and multi-concept composition.

2 Existing Work & Limitation

Much of prior work has been done in this topic. *DreamBooth* (1) fine-tunes a pretrained text-to-image diffusion model to bind a rare placeholder token to a target subject or style, often with prior-preservation regularization. It delivers strong identity fidelity and good scene transfer, but can overfit with few or homogeneous images, suffer identity drift if regularization is mis-tuned. Full-parameter updates also make multi-concept composition and sharing less convenient.

Textual Inversion (2) learns only a new text embedding while freezing the backbone, yielding a tiny and easily shareable representation that typically preserves prompt responsiveness. Its capacity is limited, however. It often struggles with fine textures, complex geometry or distinctive brand aesthetics, and may require careful prompt engineering to elicit details.

LoRA approach (3) inserts low-rank adapters in selected layers and trains only those small matrices, striking a balance between DreamBooth’s capacity and Textual Inversion’s modularity. It offers more representational power than a single learned token and remains lightweight. Nevertheless, its performance is sensitive to adapter placement, rank and scaling. Naïve configurations can cause insufficient identity fidelity or interference when stacking multiple LoRAs. Without proper regularization and augmentation, LoRA can also overfit.

From the above, the existing methods share the problem of risks over-specialization, configuration-sensitive and still prone to overfitting. Proposed method that maintains fidelity and generalization under resource constraints while preserving promptability under few-shot data is needed.

3 Method

3.1 Proposed Method

We propose a **two-stage, few-shot personalization pipeline** that fuses the strengths of DreamBooth, Textual Inversion and LoRA while avoiding their individual weaknesses.

Stage 1 – Textual Anchor.

We first run a lightweight Textual Inversion step on the 10–30 training images to learn one or two special tokens (e.g., `<brand_obj>`). Because only the token embeddings are updated, this step is quick and prevents premature over-fitting. It as well crucially locks the new concept into the model’s text space so that normal prompting remains intact.

Stage 2 – Capacity Boost with Targeted LoRA.

Next, we inject low-rank (e.g., rank = 8–16) LoRA adapters into a *small* subset of UNet cross-attention and mid-block self-attention layers. Only these adapters are trained, giving us just enough extra capacity to capture fine textures and geometry that a token alone cannot. We pair this with (i) *prior-preservation images* of the same broad class to discourage background memorization, and (ii) moderate data augmentation (crop, flip and mild color jitter) to improve generalization without distorting identity. Most of the backbone and all of the text encoder remains frozen, so the final checkpoint is composable and easy to mix with other LoRAs or control modules.

In this method, Textual Inversion secures promptability. LoRA supplies the representational power DreamBooth offers, but in a modular form. Prior-preservation and augmentation curb the over-specialization that both DreamBooth and poorly tuned LoRA often exhibit. The result generalizes beyond the tiny training set and fits comfortably within a limited training resource budget.

3.2 Problem Setup

We tackle a **few-shot personalization** task for text-to-image diffusion models. The input to the system is a tiny collection, which is typically 10–30 carefully chosen images depicting a single subject (e.g., one dog, a specific product) or a coherent visual style (e.g., a brand’s illustration look). Each image is paired with a short prompt that reserves a dedicated placeholder token, written as `<c>`, to stand for this new concept.

Starting from an off-the-shelf Stable-Diffusion–type generator, our goal is to produce a personalized version that

- **recognizes the placeholder token** and renders the target subject/style faithfully in novel scenes,
- **remains responsive to the rest of the prompt**, so users can still control pose, lighting, background, etc.,
- **generalizes beyond the training images** instead of memorizing their exact backgrounds, and
- **adds only a small number of trainable parameters**, allowing the concept to be shared or combined with others.

To mitigate over-fitting, we also draw on an auxiliary “class prior” image set representing the broad category (e.g., generic dogs) and apply light data augmentation. Next, we will formalizes these elements and specifies how the placeholder embedding and LoRA adapters are learned.

3.3 Mathematical Setup

Let

$$D = \{(x_i, y_i)\}_{i=1}^N, \quad N \in [10, 30],$$

be the *few-shot* personalization set, where each image $x_i \in \mathbb{R}^{H \times W \times 3}$ depicts the target subject or style and y_i is a prompt that contains a **placeholder token** $\langle c \rangle$ (e.g., “a photo of $\langle c \rangle$ in a park”).

We begin with a pretrained latent-diffusion model that comprises

- **VAE encoder-decoder** $(\mathcal{E}, \mathcal{D})$ mapping an image to a latent $z_0 = \mathcal{E}(x)$ and back;
- **Text encoder** $\tau(\cdot)$ producing a conditioning vector $c = \tau(y)$;
- **UNet denoiser** $\epsilon_\theta(z_t, t, c)$ parameterized by θ .

3.3.1 Diffusion Objective (frozen backbone).

A clean latent z_0 is diffused to step t by

$$z_t = \alpha_t z_0 + \sigma_t \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I),$$

with fixed schedule $\{\alpha_t, \sigma_t\}$. The standard denoising loss is

$$\mathcal{L}_{\text{diff}}(\theta) = \mathbb{E}_{(x,y), t, \varepsilon} [\|\varepsilon - \epsilon_\theta(z_t, t, \tau(y))\|_2^2].$$

For personalization we do **not** update all of θ ; instead we introduce two compact parameter sets:

- **Token embedding** $v_c \in \mathbb{R}^d$ for the new concept $\langle c \rangle$ (Stage 1);
- **LoRA adapters** $\psi = \{A_\ell, B_\ell\}$ inserted into a chosen subset of UNet projections (Stage 2).

3.3.2 Textual Inversion (Stage 1).

The text-encoder embedding matrix $W \in \mathbb{R}^{V \times d}$ is augmented with row v_c . With the backbone frozen, we optimize

$$\min_{v_c} \mathbb{E}_{(x,y), t, \varepsilon} [\|\varepsilon - \epsilon_{\theta_0}(z_t, t, \tau_{v_c}(y))\|_2^2],$$

where θ_0 denotes all pretrained weights.

3.3.3 Targeted LoRA Adapters (Stage 2).

For a linear map $W_0 \in \mathbb{R}^{m \times n}$ inside UNet (e.g., an attention projection), LoRA re-parameterizes it as

$$W = W_0 + \Delta W, \quad \Delta W = \frac{\alpha}{r} B A, \quad A \in \mathbb{R}^{r \times n}, \quad B \in \mathbb{R}^{m \times r}.$$

Only A, B are trainable; r (rank) is small (e.g., 8–16). Let ψ collect all such A, B pairs. Using the learned v_c , **Stage 2** minimizes

$$\mathcal{L}_{\text{inst}}(\psi) + \lambda_{\text{prior}} \mathcal{L}_{\text{class}}(\psi),$$

where

$$\mathcal{L}_{\text{inst}} = \mathbb{E}_{(x,y)} [\|\varepsilon - \epsilon_{\theta_0, \psi}(z_t, t, \tau_{v_c}(y))\|_2^2],$$

$$\mathcal{L}_{\text{class}} = \mathbb{E}_{(x^{(c)}, y^{(c)})} [\|\varepsilon - \epsilon_{\theta_0, \psi}(z_t^{(c)}, t, \tau(y^{(c)}))\|_2^2],$$

with $D_{\text{class}} = \{(x^{(c)}, y^{(c)})\}$ supplying *prior-preservation* images of the broader category (e.g., “a photo of a dog”). Mild augmentations $a(\cdot)$ (crop, flip, color jitter) are applied to instance images to aid generalization.

The final personalized generator is the pretrained backbone augmented by the learned token embedding v_c and the compact LoRA weights ψ ; it accepts ordinary prompts containing $\langle c \rangle$ and remains compatible with external control modules.

3.4 Model

Our pipeline maintains the **three-part architecture** of latent diffusion—VAE, text encoder, and UNet, while adding only two lightweight components: (i) a learned placeholder embedding and (ii) a set of low-rank LoRA adapters.

3.4.1 Base Generator

We adopt the public *Stable Diffusion v1.5* (4) checkpoint. The VAE encodes a 512×512 RGB image into a $64 \times 64 \times 4$ latent, the CLIP text encoder maps prompts to a 77-token sequence of 768-d vectors, and the UNet denoiser contains 860 M parameters arranged in a “down – middle – up” stack with cross-attention at each resolution.

3.4.2 Stage 1 — Placeholder Embedding

A new token row $v_c \in \mathbb{R}^{768}$ is appended to the CLIP embedding matrix. No other weights are touched. Training runs for 1 000–2 000 denoising steps with a batch size of four augmented instance images and uses the same linear noise schedule as the base model. This injects the concept into text space without harming promptability.

3.4.3 Stage 2 — Targeted LoRA Adapters

We insert LoRA blocks into every cross-attention projection (query, key, value, out) of the mid-block and the two deepest up-/down-sampling blocks, twelve projections in total.

Each LoRA is rank $r = 8$ (or 16 in ablation), scaled by $\alpha = 1$. This adds roughly 2.6 M trainable parameters ($< 0.3\%$ of UNet).

All base weights, the VAE, and the entire text encode, including the newly learned v_c , remain fixed, so only the low-rank matrices A, B receive gradients.

3.4.4 Losses and Training Mix

Each mini-batch contains an equal number of *instance* latents (with prompts that include $\langle c \rangle$) and *class prior* latents (generic category prompts without $\langle c \rangle$). The two losses share identical noise levels and are summed with weight $\lambda_{\text{prior}} = 1.0$. Instance images receive mild augmentations, random 512-px crop, horizontal flip, ± 0.05 brightness/contrast jitter to discourage background memorisation.

3.4.5 Sampling and Composition

At inference we load the base SD weights, the placeholder embedding, and the LoRA checkpoint. Both pieces are *hot-swappable*: multiple LoRAs can be merged by weighted addition, and the learned token can coexist with standard prompts or with external controls such as ControlNet or IP-Adapter, because all interface points (text space and attention tensors) are unchanged.

The result is a model that is trainable on resources provided by Datahub, yet matches DreamBooth-level fidelity while keeping the promptability and modularity of Textual Inversion and LoRA.

3.5 Algorithm

The full pipeline and pseudocode follows the notation introduced in the previous sections.

Inputs

- Few-shot concept set $D = \{(x_i, y_i)\}_{i=1}^N$ with placeholder token $\langle c \rangle$ in every prompt.
- Class-prior set D_{class} with generic category prompts (no $\langle c \rangle$).
- Pretrained Stable Diffusion v1.5 weights $(\mathcal{E}, \mathcal{D}, \tau, \theta_0)$.
- Hyper-parameters: (i) learning rates $\eta_{\text{TI}}, \eta_{\text{LoRA}}$; (ii) LoRA rank r ; (iii) prior weight λ_{prior} ; (iv) total steps $T_{\text{TI}}, T_{\text{LoRA}}$.

Procedure

1. Pre-processing

- Center-crop and resize all images to 512×512 .
- Build instance prompts $\{y_i\}$ that contain the placeholder token.
- Pre-generate class-prior images if a synthetic prior is used.

2. Stage 1 – Textual Inversion

- (a) Append a new embedding row v_c to the text-encoder vocabulary.
- (b) $\theta \leftarrow \theta_0$; freeze every parameter except v_c .
- (c) For $t = 1$ to T_{TI} :
 - Sample a mini-batch from D ; apply mild augmentation $a(\cdot)$.
 - Encode latents $z_0 = \mathcal{E}(a(x))$; pick timestep $k \sim \mathcal{U}[1, \dots, T]$.
 - Diffuse to z_k ; compute noise loss $\mathcal{L}_t = \|\varepsilon - \epsilon_\theta(z_k, k, \tau_{v_c}(y))\|_2^2$.
 - Update $v_c \leftarrow v_c - \eta_{\text{TI}} \nabla_{v_c} \mathcal{L}_t$.

3. Stage 2 – Targeted LoRA Fine-Tuning

- (a) Insert LoRA blocks (rank r) in selected UNet projections; collect all new weights in ψ .
- (b) Freeze v_c and every base weight; optimize only ψ .
- (c) For $t = 1$ to T_{LoRA} :
 - Sample half a batch from D (instance) and half from D_{class} (prior); augment the instance images.
 - For each sample, compute noise-level k , latent z_k , and corresponding loss.
 - Total loss
$$\mathcal{L}_t = \mathcal{L}_{\text{inst}}(\psi) + \lambda_{\text{prior}} \mathcal{L}_{\text{class}}(\psi).$$
 - Update $\psi \leftarrow \psi - \eta_{\text{LoRA}} \nabla_{\psi} \mathcal{L}_t$.
 - Every N_{val} steps, generate a fixed validation grid; stop early if over-fitting artefacts appear.

4. Checkpoint & Deployment

- Save the learned token embedding v_c and the LoRA state dict ψ as two small files.
- At inference time, load the base Stable Diffusion weights, apply LoRA ψ , register the token embedding v_c , and sample with arbitrary prompts that include $\langle c \rangle$. Optional control modules (ControlNet, IP-Adapter) can be attached without modification.

4 Discussion

4.1 Novelty & Advantage of the Method

As mentioned in Section 2, existing personalization techniques each solve only part of the puzzle. DreamBooth achieves strong identity fidelity but rewrites millions of weights and often degrades prompt controllability; Textual Inversion is delightfully light but lacks capacity for fine textures; LoRA strikes a middle ground yet remains vulnerable to over- or under-fitting depending on how it is configured. Our method is novel in the way it combines these three lines into a single, staged recipe: we first pin the new concept in the text space with **Textual Inversion**, then inject just-enough extra expressivity through a *minimal* set of **LoRA** adapters, all while leaving the backbone entirely frozen. This “textual anchor $\rightarrow$ targeted capacity boost” sequence is simple, but it cleanly separates the roles, as promptability is secured before any vision weights are touched, and the adapters are free to specialize without fighting the text encoder.

A second source of novelty lies in the granular placement and scale of the adapters. Rather than blanket-adding LoRAs to every attention block, we limit them to twelve key projections in the deepest layers and the mid-block, where the model captures the most important information about the subject’s appearance using a rank of only 8. This design keeps the trainable footprint below 0.3% of the UNet yet matches DreamBooth-level fidelity. Coupling this with a balanced instance/prior loss and mild augmentations yields a model that generalizes beyond the tiny training set while maintaining the subject’s distinctive features.

Moreover, the pipeline is engineered for modularity and reproducibility. Both outputs: the token embedding and the LoRA weights, are small files that can be swapped or linearly merged with other adapters, and every hyperparameter (layer list, rank, prior weight) is exposed for ablation. These choices transform previously careful hand-tuning efforts into a repeatable way that scales from limited available Datahub computing resources to production workflows without rewriting or re-training the full diffusion model.

4.2 Limitation of the Method

Although the method is lightweight and efficient, it does not solve every personalization case.

First, it still assumes that the user can supply *10 – 30 high-quality, moderately diverse images* of the target concept. When the input set is narrower, for instance, five selfies taken from the same angle, then the learned token embedding overfits quickly and the LoRA step cannot fully recover variety. Conversely, if the concept is extremely complex (e.g., a multi-part scene or an complicate pattern that never repeats exactly), the limited rank of the adapters may under-represent fine detail. In both edge cases, identity fidelity or prompt responsiveness degrades.

Second, the method relies heavily on a manually curated class-prior mix and on a few hyper-parameters (LoRA rank, prior weight, layer list). Although these hyper-parameters can be explored through ablation, they still require a small grid search, which means the method is not as easy to use as a pure Textual Inversion baseline. In addition, combining multiple LoRAs, for instance, one for a character and another for a style, may cause the adapters to interfere with each other, and this interference often needs manual adjustment of their weights.

Third, because we keep the entire backbone frozen, we inherit all biases and resolution limits of Stable Diffusion v1.5. Consequently, Large-format or aspect-ratio-extreme images may show stretching, and any demographic or stylistic bias present in the base model can surface in the personalized generator.

These limitations requires future work, like automatic hyper-parameter selection, adapter conflict resolution and partial backbone adaptation at higher resolution.

References

- [1] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman, “Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation,” 2023.
- [2] R. Gal, Y. Alaluf, Y. Atzmon, O. Patashnik, A. H. Bermano, G. Chechik, and D. Cohen-Or, “An image is worth one word: Personalizing text-to-image generation using textual inversion,” 2022.
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” 2021.
- [4] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10684–10695, June 2022.